МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Кафедра комп’ютерних наук

КУРСОВА РОБОТА

(ПОЯСНЮВАЛЬНА ЗАПИСКА)

з дисципліни Серверні технології та бекенд-розробка

на тему:

«Вебдодаток ветеринарної клініки»

студента II курсу групи КН-24-2
спеціальності 122 «Комп’ютерні науки»
Горбатюк Катерини-Вероніки Ігорівни
(прізвище, ім’я та по-батькові)

Керівник: викладач кафедри КН Віктор Желізко

Дата захисту: " 2 " червня 2026р.
Національна шкала _____
Кількість балів: _____
Оцінка: ECTS _____

Члени комісії _____ Андрій ОРИНЧАК
(підпис) (прізвище та ініціали)
_____ Віктор ЖЕЛІЗКО
(підпис) (прізвище та ініціали)
_____ Денис ФУРІХАТА
(підпис) (прізвище та ініціали)

Житомир – 2026

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення
Освітній рівень: бакалавр
Спеціальність 122 «Комп'ютерні науки»

«ЗАТВЕРДЖУЮ»
В.о. завідувача кафедри
комп'ютерних наук
Віталій ЛЕВКІВСЬКИЙ
" 10 " лютого 2026 р.

ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ
Горбатюк Катерині-Вероніці Ігорівні

1. Тема роботи: Вебдодаток ветеринарної клініки,
керівник курсового проекту: Віктор ЖЕЛІЗКО
2. Строк подання студентом: " 25 " травня 2026р.
3. Вихідні дані до роботи: Розробити CMS-систему вебдодаток ветеринарної клініки.
4. Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці)
   1. Постановка завдання
   2. Аналіз аналогічних розробок
   3. Алгоритми роботи програми
   4. Опис роботи програми
   5. Програмне дослідження

5. Перелік графічного матеріалу(з точним зазначенням обов'язкових креслень)
   1.Презентація
   2.Посилання на репозиторій: https://github.com/kn242gki-a11y/KursovaGK.git

6. Консультанти розділів проекту (роботи)

Розділ	Прізвище, ініціали та посади консультанта	Підпис, дата	
		завдання видав	завдання прийняв
1	Віктор ЖЕЛІЗКО	10.02.2026	20.02.2026
2	Віктор ЖЕЛІЗКО	20.02.2026	05.04.2026
3	Віктор ЖЕЛІЗКО	05.04.2026	20.05.2026

7. Дата видачі завдання "10" лютого 2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів курсової роботи	Строк виконання етапів проекту	Примітки
1	Постановка задачі	10.02.2026	Виконано
2	Пошук, огляд та аналіз аналогічних розробок	25.02.2026	Виконано
3	Формулювання технічного завдання	02.03.2026	Виконано
4	Опрацювання літературних джерел	09.03.2026	Виконано
5	Проектування структури	20.03.2026	Виконано
6	Написання програмного коду	05.04.2026	Виконано
7	Відлагодження	07.04.2026	Виконано
8	Написання пояснювальної записки	29.05.2026	Виконано
9	Захист	02.06.2026	Виконано

РЕФЕРАТ

Курсова робота присвячена розробці вебзастосунку для автоматизації процесів ветеринарної клініки.

Пояснювальна записка до курсової роботи на тему «Вебдодаток ветеринарної клініки» містить вступ, три основні розділи, загальні висновки, список використаної літератури та додатки.

Текстовий матеріал викладено на 33 сторінках друкованого тексту. Обсяг додатків становить 8 сторінки. Список використаних джерел охоплює 10 найменувань. Робота ілюстрована 21 рисунками та містить 4 таблиць. Повний обсяг курсової роботи 41 сторінок.

У першому розділі досліджено предметну область, розглянуто специфіку цифровізації роботи ветеринарного закладу, проведено порівняльний аналіз програмних аналогів і обґрунтовано вибір технологічного стеку розробки.

У другому розділі спроєктовано архітектуру системи за шаблоном MVC, розроблено модель реляційної бази даних із всіма таблицями та реалізовано ключові функціональні модулі вебзастосунку.

У третьому розділі висвітлено особливості реалізації користувацького інтерфейсу, а також наведено результати функціонального тестування основних користувацьких сценаріїв (облік пацієнтів, обробка термінових звернень, організація волонтерської допомоги та фінансових зборів).

Ключові слова: вебзастосунок, ветеринарна клініка, PHP, MVC, екстрений виклик.

ЗМІСТ

ВСТУП 6
РОЗДІЛ 1 АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ 8
1.1. Аналіз задачі, засобів та методів її вирішення 8
1.2. Аналіз існуючого програмного забезпечення за тематикою курсового проекту 9
Висновки до першого розділу 12
РОЗДІЛ 2 ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ 14
2.1.Проєктування бази даних інтернет-магазину рослин 14
2.2.Розробка функціональних модулів 15
2.3.Розробка програмного забезпечення 17
Висновки до другого розділу 19
РОЗДІЛ 3 ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ 21
3.1.Опис роботи з програмним додатком 21
3.2.Тестування роботи програмного забезпечення 28
Висновки до третього розділу 30
ВИСНОВКИ 31
ДОДАТКИ 34

ВСТУП

Актуальність теми. Стрімка цифровізація сфери послуг та зростаючі вимоги населення до якості сервісу зумовлюють активне впровадження сучасних інформаційних вебтехнологій у діяльність закладів ветеринарної медицини та послуг для тварин. Забезпечення оперативного надання допомоги, обробка термінових звернень, ведення електронних карт пацієнтів та організація волонтерської підтримки вимагають відходу від паперового документообігу та застарілих форматів обліку. Створення єдиного спеціалізованого вебресурсу дозволяє оптимізувати щоденну роботу персоналу, підвищити прозорість благодійних зборів та вивести на новий рівень взаємодію з власниками тварин і волонтерами, що безпосередньо підтверджує актуальність і практичну цінність обраного напряму дослідження.

Об’єктом дослідження є процес автоматизації інформаційних процесів та програмної реалізації вебсистем у сфері ветеринарного обслуговування та притулків для тварин.

Предметом дослідження є методи, засоби розробки та архітектурні шаблони, що застосовуються для проєктування та створення розширеного вебзастосунку ветеринарного закладу.

Метою роботи є проєктування та практична розробка вебзастосунку ветеринарної клініки, який поєднує користувацький функціонал (подачу екстрених заявок, викликів додому, перегляд профілів тварин і участь у благодійних зборах) та адміністративну панель (управління медичними картами, волонтерськими анкетами й обліком пацієнтів).

Для досягнення поставленої мети передбачено аналіз специфіки предметної області та дослідження наявних програмних аналогів на ринку, обґрунтування вибору оптимального технологічного стеку для розробки вебзастосунку, проєктування таблиць та архітектури майбутньої системи, програмну реалізацію функціональних модулів вебресурсу відповідно до шаблону MVC на мові PHP, а також проведення функціонального

тестування реалізованих сценаріїв взаємодії користувачів і адміністратора із системою.

РОЗДІЛ 1 АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ

1.1. Аналіз задачі, засобів та методів її вирішення

Динамічний розвиток ветеринарної медицини та сфер захисту тварин потребує створення ефективних, надійних і зручних цифрових рішень. Розробка інформаційної системи ветеринарної клініки передбачає вирішення комплексних задач: систематизацію обліку пацієнтів, збереження медичних історій і щеплень, обробку звернень, а також забезпечення взаємодії з волонтерами, підтримку благодійних зборів та дотримання вимог до безпеки й доступності даних.

В основу розробки вебзастосунку покладено архітектурний шаблон MVC, що передбачає чітке розмежування відповідальності між компонентами системи. Модель забезпечує взаємодію із СУБД та маніпуляцію даними, представлення відповідає за генерацію та відображення користувацького інтерфейсу, а контролер приймає HTTP-запити, реалізує бізнес-логіку та координує зв'язок між іншими складовими. У даній системі зазначений підхід реалізовано через базовий клас та набір спеціалізованих контролерів, таких як SettingsController, UploadController, VolunteerController, що гарантує високу модульність, масштабованість і легкість підтримки коду.

Серверна частина проєкту побудована на мові PHP з використанням кастомної MVC-архітектури. Реалізація на PHP із власною маршрутизацією дозволила досягти максимальної швидкодії та повного контролю над обробкою HTTP-сесій і запитів.

Для збереження інформації застосовано реляційну СУБД SQLite. Вона забезпечує надійне зберігання даних про тварин у таблиці animals, користувачів у users, медичні записи в health_records, а також анкети волонтерів у volunteers і фінансові збори в fundraisers. Порівняно з СУБД MySQL, яка потребує налаштування окремого сервера баз даних, SQLite має суттєві переваги: зберігання бази у єдиному файлі app.db, відсутність

потреби в окремому серверному процесі, підтримка цілісності даних через зовнішні ключі FOREIGN KEY із каскадним видаленням ON DELETE CASCADE, а також висока швидкість читання даних.

Користувацький інтерфейс реалізовано за допомогою CSS3 із використанням кастомної системи стилів, CSS-змінних та модульних сіток Flexbox і CSS Grid. На відміну від сторонніх CSS-фреймворків, зокрема Bootstrap, такий підхід мінімізує обсяг підключених ресурсів та забезпечує гнучкість у налаштуванні адаптивності й доступності. В інтерфейсі передбачено спеціальні режими доступності, включаючи клас .page-high-contrast та вибір розміру шрифту, елементи швидкої навігації skip-link.

Отже, обраний технологічний стек, що включає чистий PHP із MVC-архітектурою, SQLite через PDO, адаптивний CSS3 та HTML5, є повністю обґрунтованим, оптимізованим та забезпечує повне виконання всіх функціональних вимог до вебсистеми VetClinic.

1.2. Аналіз існуючого програмного забезпечення за тематикою курсового проекту

Перед початком розробки власного рішення доцільно проаналізувати існуючі вебсервіси ветеринарних клінік та притулків для тварин, щоб визначити їхні сильні та слабкі сторони й врахувати це при проєктуванні власного застосунку.

Ветеринарна клініка «Алден-Вет» (aldenvet.ua)

«Алден-Вет» - одна з найбільших мереж ветеринарних клінік в Україні. Сайт пропонує широкий перелік послуг, запис до лікаря, інформаційний блог та інформацію про фахівців.

Інтерфейс сайту побудований за класичною структурою корпоративного вебресурсу: головна сторінка з промо-блоками, розділи послуг, контакти та блог.

Переваги: широкий спектр висвітлених послуг, наявність інформаційних статей, зручна карта розташування філій, розгалужений перелік напрямків лікування.

Недоліки: перевантажений дизайн і складна навігація, відсутній особистий кабінет власника тварини з картою щеплень та історією хвороб, відсутній модуль екстреного виклику (Emergency) та інтеграція із благодійними зборами чи волонтерським рухом.

Ветеринарна клініка «Друзі Вет» (druzivet.com.ua)

«Друзі Вет» - житомирська ветеринарна клініка, що надає спектр діагностичних та лікувальних послуг для тварин (зокрема терапію, хірургію, УЗД, кардіологію, послуги зооготелю та виклик лікаря додому).

Інтерфейс сайту виконаний у форматі локального візитно-інформаційного вебресурсу. Навігація орієнтована на ознайомлення відвідувачів із переліком послуг, прайс-листом, контактами та графіком роботи клініки.

Переваги: наявність чітко структурованого прайс-листа, висвітлення послуг виклику лікаря додому та зооготелю, зручне розміщення контактних даних і адрес філій.

Недоліки: застарілий та спрощений дизайн, відсутність форми автоматизованого онлайн-запису на прийом (запис здійснюється лише за номерами телефонів), відсутність особистого кабінету власника тварини з електронною медичною картою та календарем щеплень, відсутні модулі для обробки екстрених звернень (Emergency) та інтеграції із благодійними зборами коштів чи анкетами волонтерів для допомоги безпритульним тваринам.

Ветеринарна клініка «ВетЛайф» (vetlifezt.wordpress.com)

«ВетЛайф» - локальна ветеринарна клініка в м. Житомир. Сайт пропонує ознайомчу інформацію про заклад, перелік послуг, графік роботи, ціни та контактні дані.

Інтерфейс сайту побудований за класичною структурою блогу на платформі WordPress: вітальний блок із контактами, розділи спеціалізації клініки (терапія, хірургія, кардіологія, репродуктологія), прайс-лист та фотоблог.

Переваги: наявність чітко структурованого переліку послуг та цін, інформація про спеціалізацію лікарів, наявність фотогалереї, детальні дані про режим роботи та контакти.

Недоліки: відсутність онлайн-форми запису на прийом (запис здійснюється лише за телефоном), відсутній особистий кабінет власника тварини з картою щеплень та історією хвороб, відсутній модуль екстреного виклику (Emergency) та інтеграція із благодійними зборами чи волонтерським рухом.

Ветеринарний центр «VetCenter» (vetcenter.in.ua)

«VetCenter» - центр ветеринарної допомоги, що надає спектр лікувально-діагностичних послуг, має власний стаціонар, готель для тварин, зоомагазин та напрямок дерматології.

Інтерфейс сайту виконаний за висвітлювальною структурою локального закладу: головна сторінка з описом основних напрямків діяльності, відкритим прайс-листом на послуги (кастрація, УЗД, лабораторна діагностика, вакцинація тощо) та контактними даними.

Переваги: наявність прозорого прайс-листу з чіткою градацією цін за вагою тварини, широкий перелік медичних та супутніх послуг

(зокрема стаціонар і готель для тварин), наочне висвітлення контактної інформації та графіка роботи.

Недоліки: відсутність онлайн-запису на прийом або виклик лікаря додому, відсутність особистого кабінету користувача для відстеження картки щеплень й історії хвороб, відсутність функцій для залучення волонтерів, подачі екстрених заявок (Emergency) та підтримки благодійних зборів коштів на лікування безпритульних тварин.

Підсумок досліджених даних наведено у таблиці 1.1.

Таблиця 1.1. Порівняльна таблиця вебсистем ветеринарної тематики

Функціональна можливість	**“Алден-Вет”**	**“Друзі Вет”**	**“ВетЛайф”**	**“VetCenter»”**	**Розроблена система**
Каталог тварин / пацієнтів	+	-	-	-	+
Електронні медичні карти та журнал вакцинації	-	-	-	-	+
Модуль екстрених заявок	-	-	-	-	+
Запис на виклик лікаря додому	-	+	-	-	+
Модуль волонтерства та зборів коштів	-	-	-	-	+
Особистий кабінет користувача	-	-	-	-	+
Налаштування доступності	-	-	-	-	+

Проведений аналіз показав, що більшість існуючих рішень чітко розмежовані: вони є або суто комерційними сайтами ветеринарних клінік, для тварин.

Спільним недоліком клінічних аналогів є відсутність особистого кабінету власника тварини з можливістю перегляду історії хвороб, журналу вакцинацій та виклику лікаря додому.

Розроблена система ветеринарної клініки усуває всі зазначені недоліки, поєднуючи в єдиній платформі функціонал ветеринарної клініки (облік пацієнтів, виклик додому, тощо) та спеціальні налаштування доступності інтерфейсу.

Висновки до першого розділу

У першому розділі було проаналізовано основні методи, засоби та архітектурні підходи до розробки сучасних вебзастосунків. Визначено, що архітектурний шаблон MVC є найбільш доцільним підходом для розробки

вебсистеми ветеринарної клініки, оскільки забезпечує чітке розмежування відповідальності між компонентами системи, підвищує безпеку обробки даних та спрощує подальше розширення коду. Для реалізації проєкту обрано легковажну кастомну MVC-архітектуру на мові PHP без використання надлишкових сторонніх фреймворків.

Проведений аналіз існуючих аналогів показав, що всі вони є вузькоспеціалізованими та мають спільні суттєві недоліки: відсутність єдиної системи, яка б поєднувала медичний облік із соціальними ініціативами, відсутність особистого кабінету власника тварини та історією хвороб, відсутність зручних модулів для екстрених викликів, подачі заявок на візит додому, а також повна відсутність спеціальних налаштувань доступності для користувачів. Це підтверджує актуальність та доцільність розробки власного комплексно інтегрованого рішення.

На основі проведеного аналізу сформовано вимоги до системи та обрано оптимальний технологічний стек для реалізації проєкту: PHP (MVC), СУБД SQLite із підтримкою стандартів доступності. Такий вибір забезпечує високу швидкість роботи застосунку, надійне збереження структурованих даних, зручність користування та повноцінний функціонал для відвідувачів, волонтерів і ветеринарного персоналу.

РОЗДІЛ 2 ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1. Проєктування бази даних Вебдодатку ветеринарної клініки

База даних вебзастосунку ветеринарної клініки “Добра лапа” реалізована засобами СУБД SQLite в середовищі Visual Studio Code та розроблена з урахуванням вимог третьої нормальної форми (3NF), що забезпечує відсутність дублювання даних і цілісність зв'язків через зовнішні ключі (FOREIGN KEY).

Загальна схема охоплює вісім таблиць: users (користувачі та адміністратори), animals (пацієнти та підопічні притулку), health_records (медичні карти), emergency_requests (екстрені виклики), home_visits (виклики ветеринара додому), volunteers (анкети волонтерів) та fundraisers (благодійні збори коштів).

Нижче наведено структури основних таблиць системи.

Таблиця 2.1 - Специфікація полів таблиці users

Назва поля	Назва поля в БД	Тип поля
Ідентифікатор	id	INTEGER PRIMARY KEY AUTOINCREMENT
Логін / Пароль	login / password	VARCHAR(50), NOT NULL, UNIQUE / VARCHAR(255), NOT NULL
Email / Телефон	email / phone	VARCHAR(100), NOT NULL / VARCHAR(20), DEFAULT ''
ПІБ (Ім'я / Прізвище)	first_name / last_name	VARCHAR(50), NOT NULL / VARCHAR(50), NOT NULL
Місто / Стать / Про себе	city / gender / about	VARCHAR(50) / VARCHAR(10) / TEXT, DEFAULT ''
Роль / Дата створення	role / created_at	VARCHAR(20), DEFAULT 'user' / DATETIME, DEFAULT CURRENT_TIMESTAMP

Таблиця animals (таблиця 2.2) є центральною сутністю системи. Вона пов'язана з таблицею users зовнішнім ключем owner_id. При видаленні облікового запису власника значення owner_id змінюється на NULL (ON DELETE SET NULL), що дозволяє зберегти картку тварини в базі притулку.

Таблиця 2.2 - Специфікація полів таблиці animals

Назва поля	Назва поля в БД	Тип поля
Ідентифікатор	id	INTEGER PRIMARY KEY AUTOINCREMENT
Кличка / Вид / Порода	name / species / breed	VARCHAR(100), NOT NULL / VARCHAR(50), NOT NULL / VARCHAR(100)
Вік / Вага (кг)	age / weight	INTEGER, DEFAULT 0 / DECIMAL(5,2), DEFAULT 0
Власник / ID власника	owner / owner_id	VARCHAR(150), NOT NULL / INTEGER, FK → users.id (ON DELETE SET NULL)
Мікрочип / Стан здоров'я	microchip / health_status	VARCHAR(50), DEFAULT '' / VARCHAR(50), DEFAULT 'здорова'
Останній візит / Створено	last_visit / created_at	DATE / DATETIME, DEFAULT CURRENT_TIMESTAMP

Таблиця health_records (таблиця 2.3) містить записи про огляди та лікування. Поле animal_id забезпечує каскадне видалення записів (ON DELETE CASCADE) у разі вилучення картки тварини.

Таблиця 2.3 - Специфікація полів таблиці health_records

Назва поля	Назва поля в БД	Тип поля
Ідентифікатор	id	INTEGER PRIMARY KEY AUTOINCREMENT
Ідентифікатор тварини	animal_id	INTEGER, NOT NULL, FK → animals.id (ON DELETE CASCADE)
Дата візиту / Діагноз	visit_date / diagnosis	DATE, NOT NULL / TEXT, DEFAULT ''
Лікування / Нотатки	treatment / vet_notes	TEXT, DEFAULT '' / TEXT, DEFAULT ''
Дата створення	created_at	DATETIME, DEFAULT CURRENT_TIMESTAMP

2.2. Розробка функціональних модулів

Вебзастосунок ветеринарної клініки “Добра лапа” реалізований за архітектурним шаблоном MVC (Model-View-Controller) мовою PHP і складається з восьми функціональних модулів, кожен із яких відповідає за окремий напрямок роботи ветеринарної клініки та притулку.

Модуль автентифікації та профілів забезпечує розмежування доступу між двома типами користувачів: адміністраторами та клієнтами/волонтерами. Доступ регулюється за допомогою сесій та поля

role у таблиці users. Для реєстрації нового користувача виконується валідація даних: перевіряється унікальність логіну та email, коректність телефонного номера за допомогою регулярних виразів, а також збіг паролів. Пароль зберігається у базі даних у вигляді хешу, згенерованого за допомогою безпечного алгоритму password_hash(). Після автентифікації користувач отримує доступ до особистого кабінету, де може редагувати дані профілю.

Модуль обліку тварин та пацієнтів є центральним у системі та реалізований у AnimalController. Модуль дозволяє ведення облікового реєстру як для домашніх улюбленців клієнтів, так і для підопічних притулку. Для кожної тварини фіксуються її кличка, вид, порода, вік, вага, стан здоров'я, номер мікрочипа та зв'язок із власником через owner_id. У разі видалення акаунту власника запис про тварину залишається в системі завдяки зв'язку ON DELETE SET NULL, що дозволяє притулку зберегти медичну історію тварини.

Модуль медичної карти та оглядів працює на основі таблиці health_records і дозволяє ветеринарним лікарям фіксувати результати прийомів.

Модуль вакцинації реалізований для відстеження календаря щеплень тварин.

Модуль екстрених викликів (EmergencyController) призначений для фіксації термінових звернень у критичних ситуаціях. Через спеціальну форму на сайті власник тварини може швидко надіслати заявку з описом симптоматики, вказавши рівень терміновості (urgency). Нові звернення отримують статус new і відображаються у пріоритетному списку панелі адміністратора для оперативного реагування.

Модуль виклику ветеринара додому реалізований для обробки планових виїздів спеціалістів за адресою клієнта. Форма заявки передбачає введення адреси, контактного телефону, бажаної дати та часу візиту. Адміністратор системи має можливість змінювати статус заявки (pending,

confirmed, completed) та розподіляти виклики між черговими лікарями.

Модуль волонтерства забезпечує роботу з кандидатами та активними помічниками притулку. Охочі допомогти тваринам залишають анкету із зазначенням бажаного напрямку діяльності (догляд, вигул, транспортування, фотозйомка), описом досвіду та доступного графіку. Заявка отримує початковий статус pending та потрапляє на розгляд адміністратору.

Модуль фінансових зборів (фандрайзинг) призначений для залучення коштів на операції, складне лікування та закупівлю кормів для тварин притулку. Модуль відображає прогрес збору у вигляді співвідношення зібраної суми (collected_amount) до цільової (target_amount). При досягненні 100% суми статус збору автоматично або вручну змінюється на completed.

Адміністративна панель захищена перевіркою прав доступу (middleware/перевірка сесії на role === 'admin'). Вона надає адміністраторам повний спектр CRUD-операцій (створення, читання, оновлення, видалення) для всіх 8 таблиць бази даних: керування користувачами, картками тварин, призначеннями, обробка екстрених звернень та редагування фінансових зборів. Таблиці даних у панелі оснащені зручним фільтром та сортуванням.

Центральними сутностями є users та animals. Таблиці health_records та vaccinations пов'язані з animals зв'язком «один до багатьох» (одна тварина може мати багато медичних записів та щеплень) із каскадним видаленням. Таблиця animals пов'язана з users через owner_id зв'язком «один до багатьох» із можливістю присвоєння значення NULL. Таблиці emergency_requests, home_visits, volunteers та fundraisers є автономними модульним сутностями, що зберігають оперативні дані та заявки користувачів.

2.3. Розробка програмного забезпечення

Розробка програмного забезпечення виконувалася поетапно відповідно до обраної MVC-архітектури. На першому етапі в середовищі Visual Studio Code було налаштовано базову структуру проєкту, маршрутизацію (Router) та підключено СУБД SQLite через об'єктно-

орієнтований інтерфейс PDO. На другому етапі розроблено та виконано SQL-скрипти створення восьми таблиць бази даних із налаштуванням первинних і зовнішніх ключів. На третьому етапі реалізовано базовий контролер, класи моделей, класи проміжного програмного забезпечення (middleware) та HTML/CSS-шаблони для кожного з функціональних модулів системи.

Ключові функціональні класи системи реалізують обробку HTTP-запитів, валідацію вхідних даних та взаємодію з базою даних:

Метод AnimalController@index обробляє GET-запит до сторінки списку тварин і реалізує динамічну фільтрацію та пошук пацієнтів. Пошук виконується за допомогою SQL-оператора LIKE одночасно за кличкою тварини, видом та породою з підтримкою часткового збігу. Для отримання даних про власника виконується об'єднання таблиць animals та users через LEFT JOIN. Це дозволяє отримати повну інформацію про тварину та її власника за один SQL-запит без виникнення проблеми $N+1$. Лістинг методу наведений у додатку В.

Метод EmergencyController@store відповідає за обробку форми екстреного звернення. Метод виконує первинну валідацію вхідних даних: перевіряє заповненість обов'язкових полів (ПІБ власника, телефон, кличка тварини, опис проблеми) та валідує формат телефонного номера за допомогою регулярного виразу. Після успішної перевірки формується новий запис у таблиці emergency_requests зі статусом new та поточним часовим штампом CURRENT_TIMESTAMP. Метод підтримує повернення JSON-відповіді для AJAX-запитів, що дозволяє відображати повідомлення про успішне відправлення без перезавантаження сторінки. Лістинг методу наведений у додатку В.

Клас Animal реалізує взаємодію з таблицею animals через об'єкт PDO. Модель містить методи для обчислення додаткових характеристик пацієнта. Метод getMedicalHistory() повертає масив усіх оглядів та щеплень тварини, об'єднуючи дані з таблиць health_records та vaccinations за ідентифікатором

animal_id. Метод getStatusBadge() повертає відповідний CSS-клас для візуального відображення стану здоров'я тварини в інтерфейсі (наприклад, «здорова», «на лікуванні», «потребує огляду»).

Лістинг наведений у додатку В.

Авторизація та обмеження доступу реалізовані за допомогою компонента AuthMiddleware. Під час кожного HTTP-запиту middleware перевіряє наявність активної сесії ($_SESSION['user_id']).

Метод checkAdmin() перевіряє значення поля role у поточного користувача. Якщо користувач із роллю user намагається отримати доступ до маршрутів адміністративної панелі (/admin/*), виконується автоматичний редирект на сторінку авторизації з відображенням повідомлення про обмеження доступу. Такий підхід забезпечує надійний захист персональних даних клієнтів та медичних карт. Лістинг методів наведений у додатку В.

Висновки до другого розділу

У другому розділі виконано проєктування та програмну реалізацію вебзастосунку ветеринарної клініки та притулку «Добра лапа». Засобами СУБД SQLite в середовищі Visual Studio Code спроєктовано та створено реляційну базу даних, що відповідає вимогам третьої нормальної форми (3NF). Схема охоплює вісім взаємопов'язаних таблиць: users, animals, health_records, vaccinations, emergency_requests, home_visits, volunteers та fundraisers. Реалізовано цілісність даних через зовнішні ключі: для таблиці animals встановлено зв'язок із users через owner_id зі значенням ON DELETE SET NULL, що дозволяє зберігати історію пацієнта в притулку при вилученні акаунта власника, а для health_records та vaccinations — каскадне видалення (ON DELETE CASCADE).

Відповідно до архітектурного шаблону MVC (Model-View-Controller) мовою PHP розроблено вісім основних модулів: автентифікації та профілів із хешуванням паролів за допомогою password_hash(), обліку тварин та пацієнтів, медичних карт, вакцинації, обробки екстрених викликів, запису на виїзд ветеринара додому, анкетування волонтерів та фандрайзингу.

Також реалізовано адміністративну панель із повним спектром CRUD-операцій, фільтрацією та сортуванням записів.

Побудовано поетапний процес розробки із застосуванням інтерфейсу PDO для безпечної взаємодії з SQLite та компонента Router для маршрутизації HTTP-запитів. Описано та реалізовано ключову бізнес-логіку системи, зокрема метод AnimalController@index, який здійснює оптимізовану фільтрацію та пошук пацієнтів через LEFT JOIN, метод EmergencyController@store, що забезпечує валідацію форм і підтримку AJAX/JSON-відповідей для термінових звернень, модель Animal для формування агрегованої історії хвороби (getMedicalHistory()) та компонент AuthMiddleware для перевірки сесій і обмеження доступу до адміністративного розділу (checkAdmin()). Розроблене програмне забезпечення повністю відповідає технічним вимогам, гарантує захист персональних даних і забезпечує зручний інструментарій для клієнтів, лікарів, волонтерів та адміністраторів ветеринарної клініки та притулку «Добра лапа».

РОЗДІЛ 3 ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ

3.1. Опис роботи з програмним додатком

При переході на вебзастосунок користувачу відкривається головна сторінка. У верхній правій частині навігаційного меню розташовані кнопки для входу та реєстрації в системі. Також у шапці сайту міститься перелік усіх доступних послуг клініки, зокрема онлайн-запис до лікаря та виклик спеціаліста додому. Нижче розміщено інформаційний блок із ключовими відомостями про заклад: графіком роботи, адресою та контактним номером телефону.

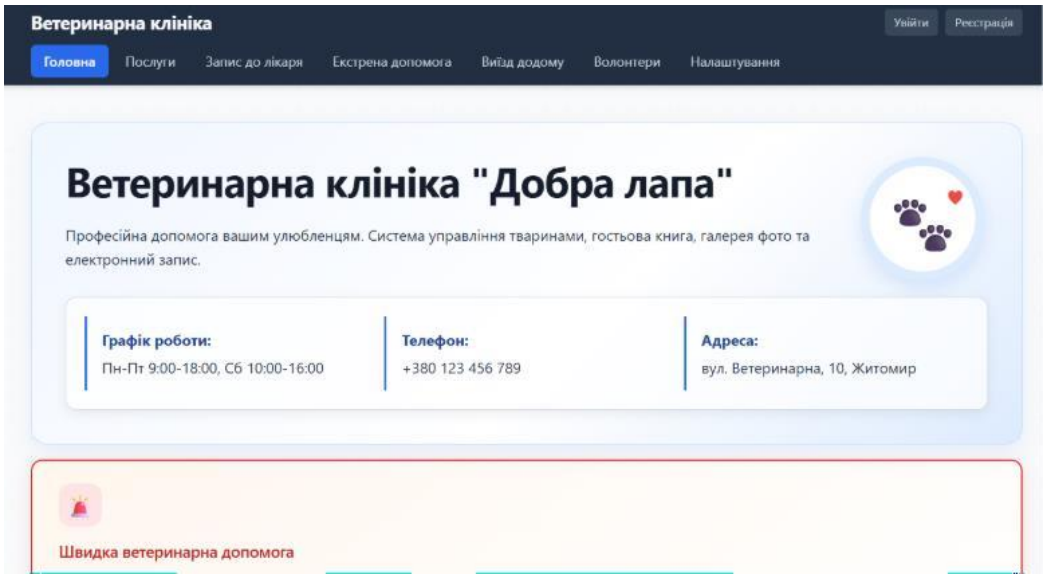


Рис.3.1.1. Головна сторінка

Окремим елементом головної сторінки є кнопка екстреної допомоги (рис.3.1.2.), розташована у центральній секції інтерфейсу. Дана функціональність доступна всім відвідувачам вебзастосунку без необхідності попередньої реєстрації чи авторизації в системі, що забезпечує оперативність зв'язку із черговим спеціалістом у термінових ситуаціях.

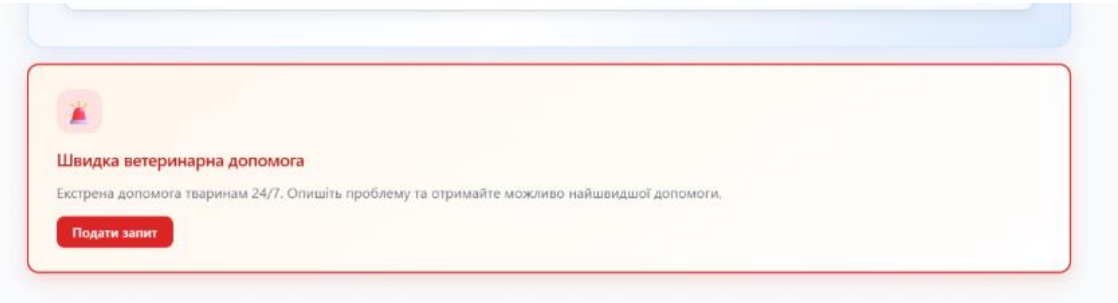


Рис.3.1.2. Кнопка швидкої ветеренароної допомоги

Вікно введення даних для швидкого реагування, що відкривається після натискання кнопки Подати заявку. Містить поля для вказування імені користувача, контактного номера телефону та поточного стану тварини. Після заповнення основної інформації, заявку буде подано.

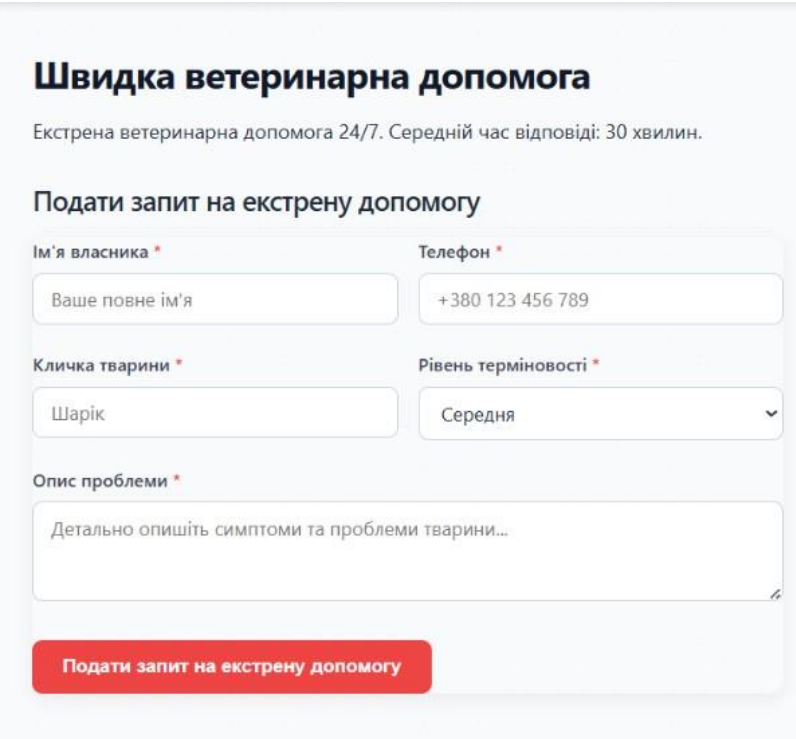


Рис.3.1.3. Вікно швидкого реагування

Доступ до всіх інших функціональних вкладок системи вимагає обов'язкової авторизації користувача (підтвердження облікового запису). Процеси оформлення різних видів планових послуг та записів мають стандартний характер: інтерфейси форм побудовані за єдиним шаблоном і містять подібний набір полів, що забезпечує зручність та цілісність користувацького досвіду (рис.3.1.4.).

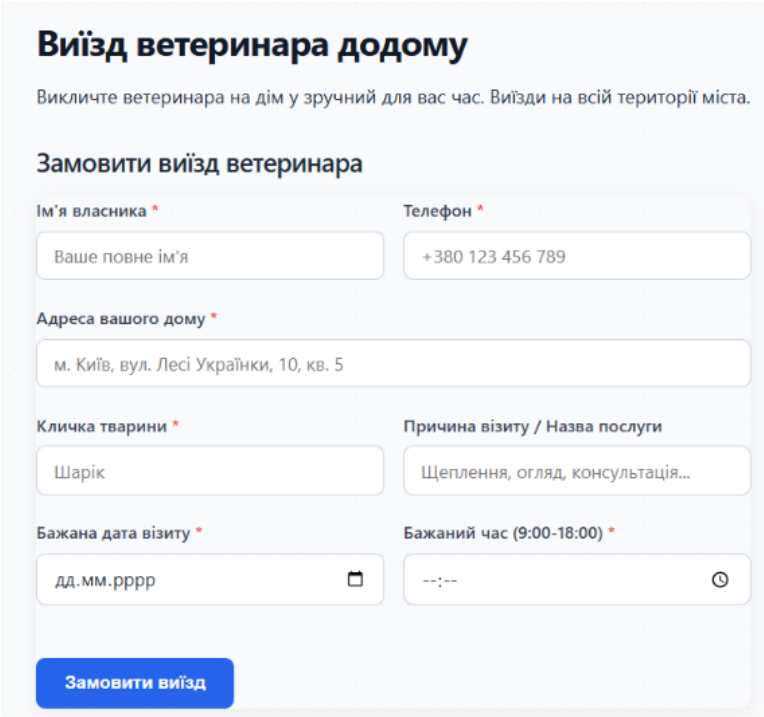


Рис.3.1.4. Шаблон запису

У вкладці Наші послуги представлено повний перелік ветеринарної допомоги, яку надає клініка (рис.3.1.5.). Для кожної позиції каталогу відображається назва послуги, короткий текстовий опис її змісту, а також фіксована вартість, що дозволяє користувачеві попередньо ознайомитися з прайсом на послуги.

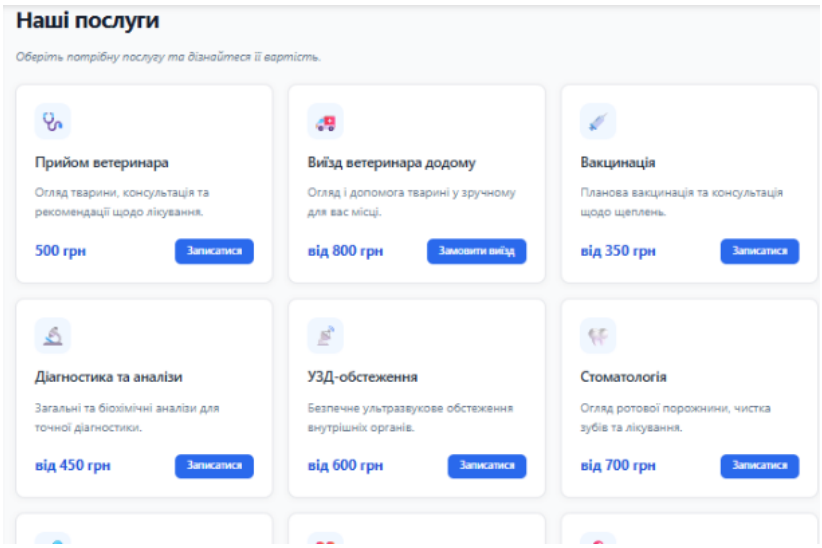


Рис.3.1.5. Вкладка послуг

На сторінці Наші волонтери (рис.3.1.6) розміщено збори коштів на лікування тварин та список волонтерів. У картках зборів видно опис потреби, зібрану суму, прогрес та кнопку Допомогти. Нижче знаходиться

кнопка Стати волонтером і картки учасників команди з їхніми контактами.

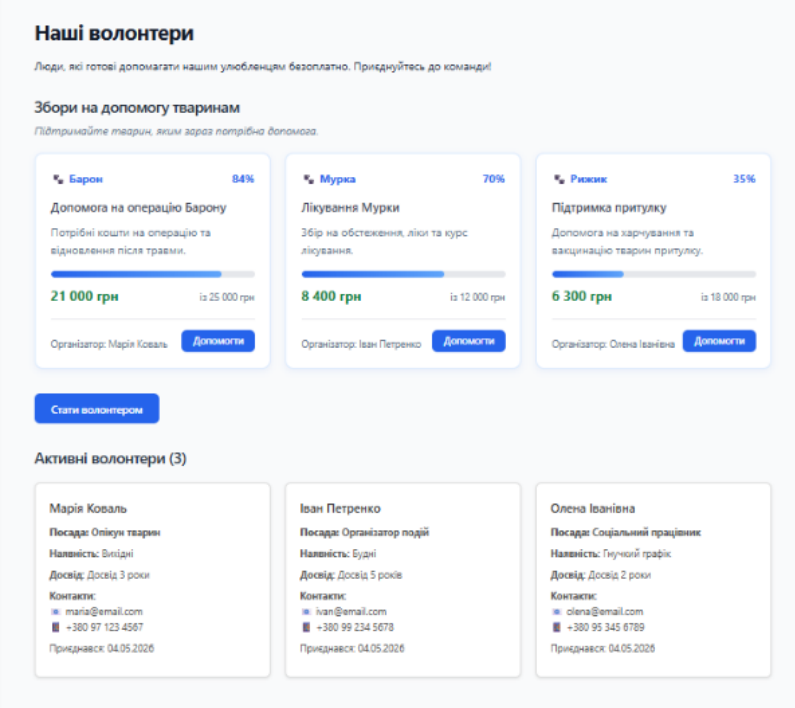


Рис.3.1.6. Вкладка волонтерства

При натисканні на кнопку Допомогти відкривається сторінка Допомогти тварині (рис.3.1.7.). На ній відображається назва збору, його короткий опис та поточний прогрес зібраних коштів. Нижче розташоване поле для введення бажаної суми донату, а також кнопки Підтвердити допомогу для переходу до оплати та Скасувати для повернення назад.

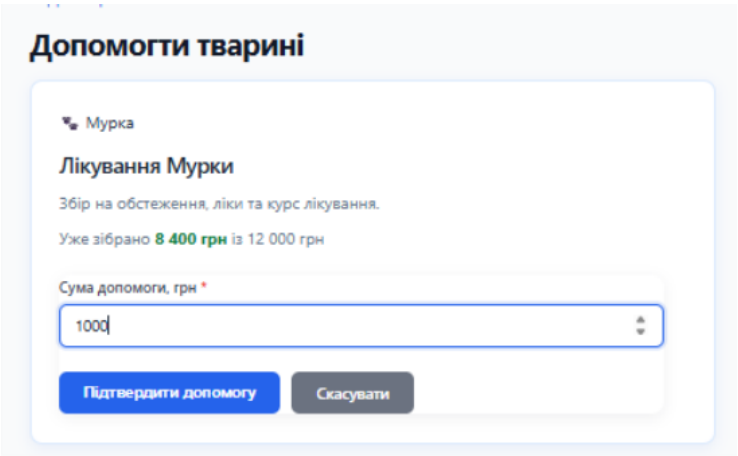


Рис.3.1.7. Вкладка з донатом

Після успішного внесення пожертви сума зібраних коштів та відсоток виконання збору автоматично оновлюються на сторінці (рис.3.1.8.). Це дозволяє користувачеві одразу побачити актуальний стан фінансування та

свій внесок у допомогу тварині.

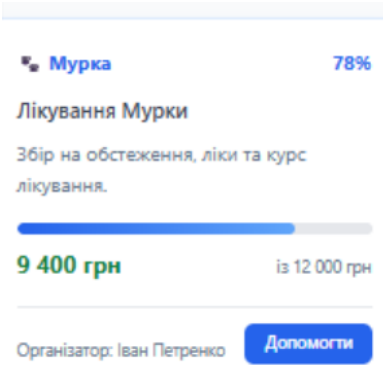


Рис.3.1.8. Збільшення відсотку збору

У вкладці Налаштування (рис.3.1.9.) користувач може адаптувати зовнішній вигляд інтерфейсу під власні побажання. Для цього передбачені параметри зміни кольорової теми сторінки, налаштування розміру шрифту та регулювання рівню його контрастності, що покращує читабельність та забезпечує зручність використання додатка.

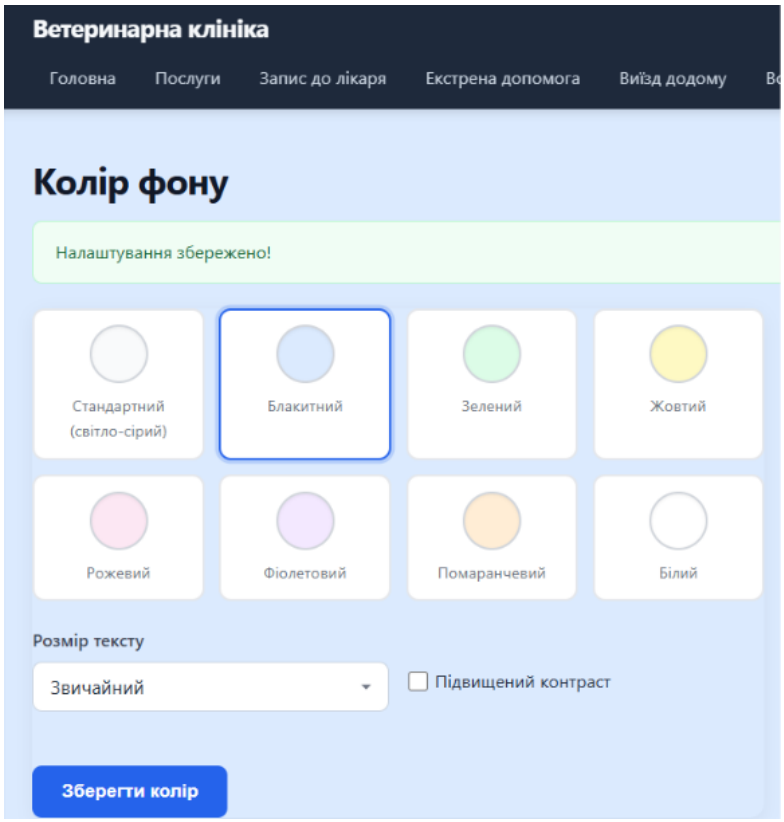


Рис.3.1.9. Вкладка налаштування інтерфейсу

Для отримання доступу до більшості функцій вебзастосунку користувачеві необхідно пройти процедуру реєстрації або авторизуватися під власним обліковим записом (рис.3.1.10-11.). Без авторизації система

надає лише обмежений перегляд базової інформації та доступ до модуля екстреної допомоги.

Рис.3.1.10. Реєстрація на сайт

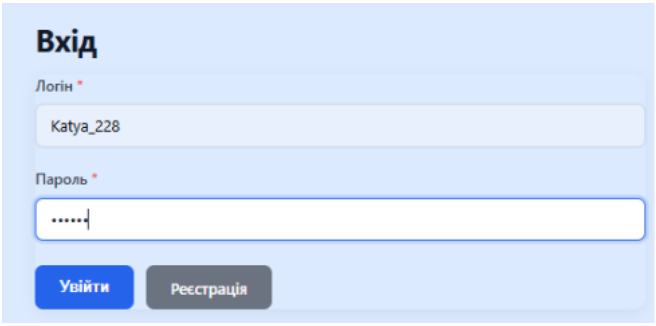


Рис.3.1.11. Вхід в обліковий запис

Після успішної авторизації в системі користувачу відкривається персональна анкета (рис.3.1.12.). У даному розділі відображаються введені під час реєстрації дані з можливістю їх редагування, виходу з облікового запису або повного видалення власного профілю із системи.

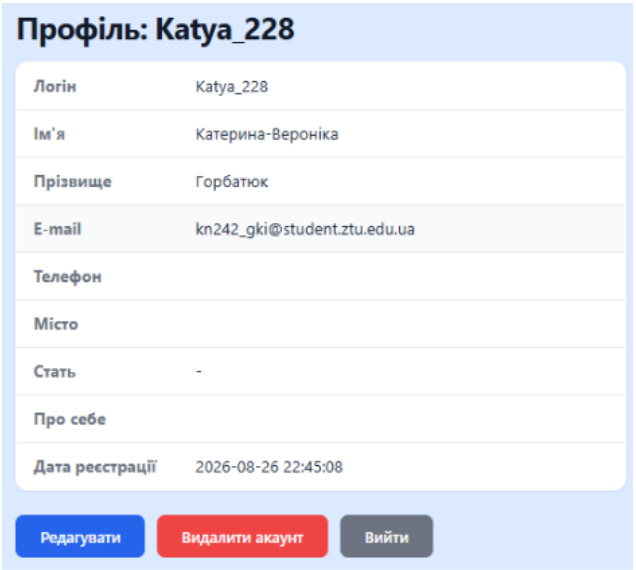


Рис.3.1.12. Профіль користувача

При авторизації під обліковим записом адміністратора в головному меню відображається додаткова вкладка Тварини (рис.3.1.13.). Цей панельний розділ призначений для розширеного управління базою даних пацієнтів та дозволяє вести облік карт тварин, переглядати історію хвороби й коригувати інформацію.

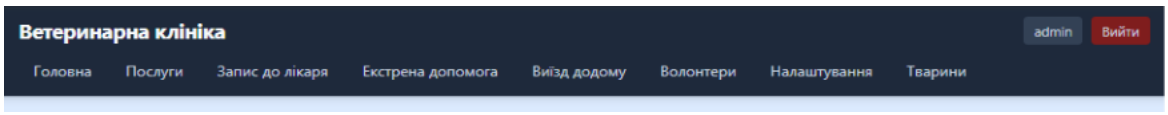


Рис.3.1.13. Панель адміністратора

У вкладці Тварини адміністратору відображається повний перелік пацієнтів, які наразі перебувають на стаціонарному лікуванні в клініці (рис.3.1.14.). Розділ забезпечує зручний моніторинг стану кожного улюбленця та дозволяє адміністратору оперативно відстежувати інформацію про тварин у стаціонарі.

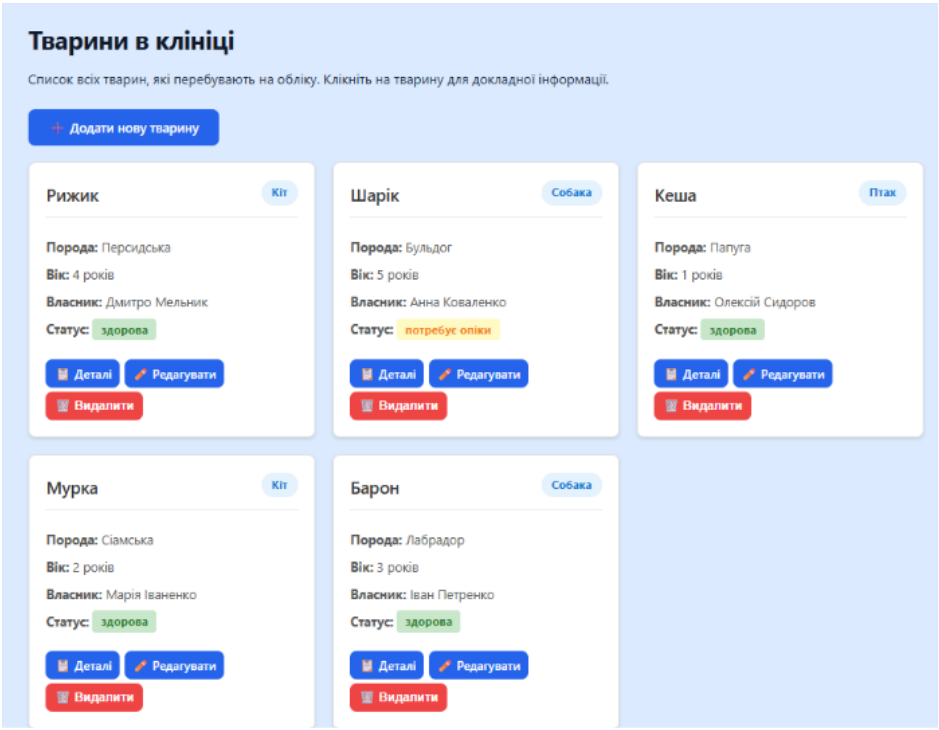


Рис.3.1.14. Вкладка тварин у стаціонарі

Окрім списку стаціонарних пацієнтів, у аккаунта адміністратора відображаються всі поточні записи на планове лікування та надходження заявок на екстрену допомогу (рис.3.1.15.). Це дозволяє адміністратору централізовано контролювати потік клієнтів, оперативно реагувати на термінові звернення та коригувати робочий графік лікарів у режимі реального часу.

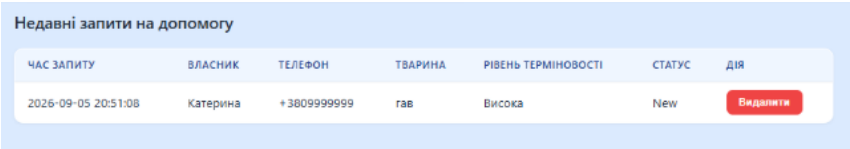


Рис.3.1.15. Заапис до лікаря

3.2. Тестування роботи програмного забезпечення

Процес реєстрації нового користувача передбачає обов'язкове заповнення базового набору полів (логін, пароль, електронна пошта, ім'я та прізвище). У випадку спроби відправки форми з незаповненими обов'язковими полями спрацьовує механізм валідації даних (рис.3.2.1.). Система блокує відправку запиту, підсвічує відповідні текстові поля червоним кольором та виводить сповіщення з переліком помилок, які необхідно виправити для успішного створення облікового запису.

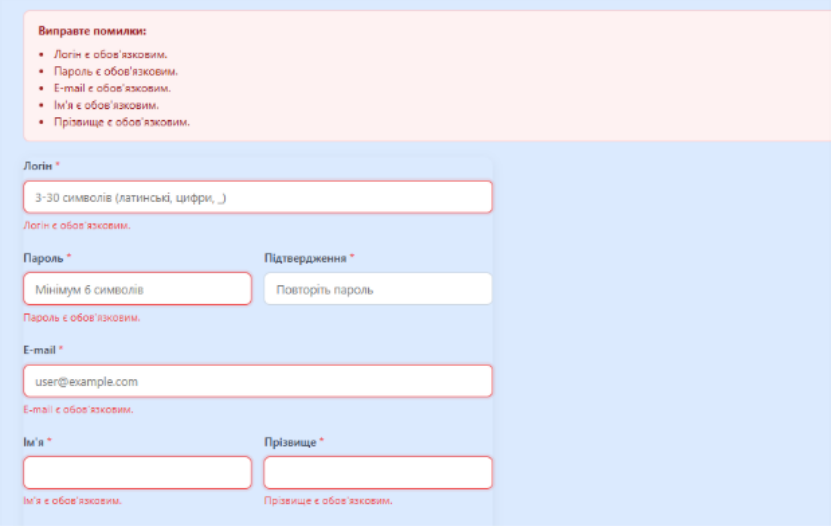


Рис.3.2.1.Помилки під час реєстрації.

Для забезпечення коректності бронювання у системі реалізовано алгоритм перевірки зайнятості часових слотів. У випадку, якщо два користувачі намагаються обрати один і той самий час та дату прийому, система успішно реєструє заявку першого користувача. Для іншого користувача, який надсилає запит пізніше, спрацьовує перевірка конфлікту розкладу: додаток виводить сповіщення про те, що обраний час вже зайнятий, та пропонує обрати альтернативну дату або вільний часовий інтервал (рис.3.2.3.).

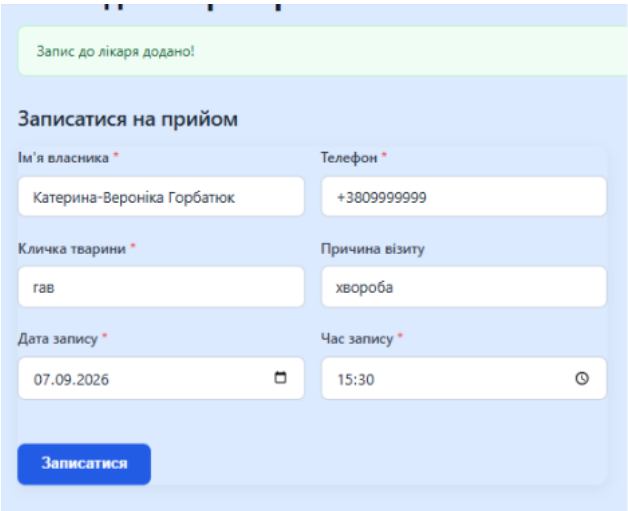


Рис.3.2.2. Запис на прийом

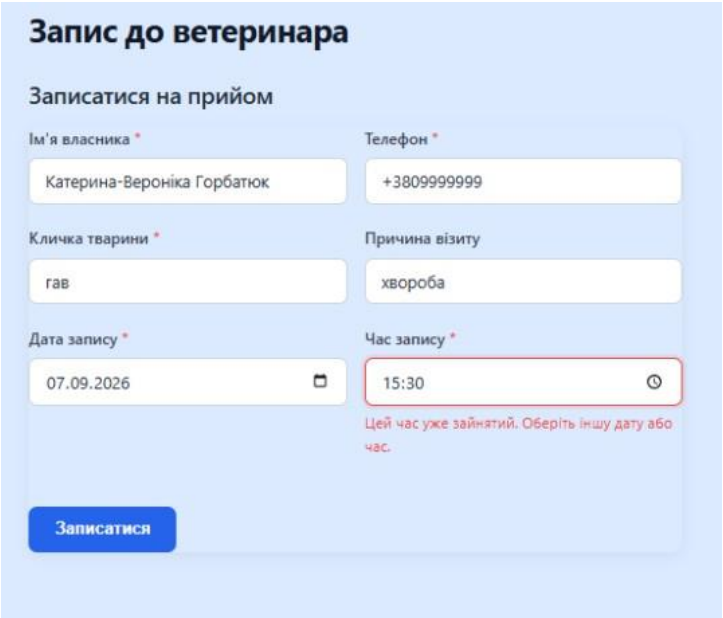


Рис.3.2.3. Сповіщення, що обраний час вже зайнятий

Реєстр записів із фіксацією лише першої успішно підтвердженої заявки на конкретний часовий слот.

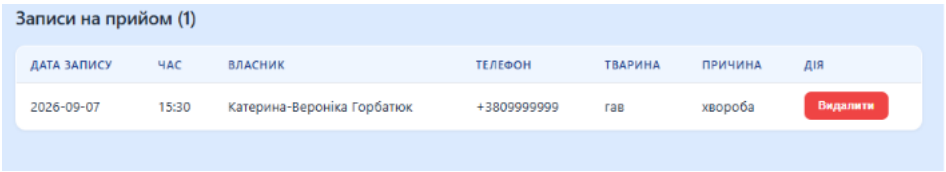


Рис.3.2.4. Вкладка запису, яка доступна лише адміністратору

Система також здійснює автоматичну перевірку відповідності обраного часу графіку роботи закладу (рис.3.2.5). Спроба онлайн-запису на години, коли клініка зачинена, блокується з відображенням відповідного повідомлення про помилку. Описане обмеження стосується лише планових

візитів і свідомо не поширюється на модуль екстрених викликів, що забезпечує можливість цілодобового оформлення термінових заявок.

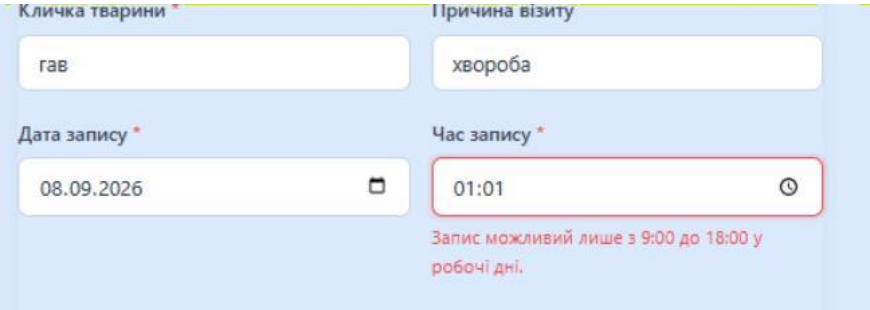


Рис.3.2.5. Повідомлення про помилку

Висновки до третього розділу

У третьому розділі висвітлено особливості реалізації інтерфейсу вебзастосунку ветеринарної клініки “Добра лапа” та порядок взаємодії з ним клієнтів, волонтерів й адміністраторів. Описано ключові сценарії використання: перегляд та фільтрацію профілів тварин, подачу екстрених звернень, запис на візит додому, ведення медичних карт й анкет волонтерів, підтримку благодійних зборів та роботу з налаштуваннями доступності.

Проведено функціональне тестування основних модулів системи. Перевірено валідацію вхідних даних у формах (формати телефонів, обов'язкові поля), коректність запису даних у SQLite, роботу зв'язків із каскадним видаленням, а також захист адміністративних маршрутів через AuthMiddleware.

За результатами тестування підтверджено повністю коректну та стабільну роботу всіх функцій системи. Розроблений вебзастосунок ветеринарної клініки “Добра лапа” є готовим до практичного використання персоналом клініки та відвідувачами.

ВИСНОВКИ

У ході виконання курсової роботи розроблено повнофункціональний вебзастосунок ветеринарної клініки “Добра лапа” мовою PHP із використанням SQLite та архітектурного шаблону MVC. Застосунок реалізує всі функціональні можливості сучасної системи обліку та адміністративного управління клінікою і складається з двох частин: публічного сайту для клієнтів та волонтерів і адміністративної панелі для ветеринарних лікарів та менеджерів притулку.

У процесі роботи виконано аналіз існуючих рішень у сфері автоматизації ветеринарних послуг для тварин. Встановлено, що більшість аналогів мають обмежений функціонал, не поєднують клінічні послуги із функціоналом фандрайзингу та волонтерства, а також не надають зручного механізму для екстреного звернення у критичних ситуаціях. Розроблений застосунок вебзастосунок ветеринарної клініки “Добра лапа” усуває всі виявлені недоліки.

Спроєктовано реляційну базу даних із основних таблиць: users, animals, health_records, vaccinations, emergency_requests, home_visits, volunteers та fundraisers. Між таблицями встановлено зв'язки через зовнішні ключі, що забезпечує цілісність даних. Налаштування зв'язків дозволяє зберігати історію пацієнтів.

Окрему увагу приділено безпеці застосунку. Паролі користувачів зберігаються виключно у вигляді безпечного хешу. Адміністративні маршрути захищені проміжним програмним забезпеченням AuthMiddleware з перевіркою ролей (role === 'admin'). Форми оснащені валідацією вхідних даних для перевірки коректності контактних телефонів, дат та обов'язкових полів.

Проведено функціональне тестування основних сценаріїв роботи системи, включаючи валідацію форм, перевірку захисту адміністративних маршрутів, обробку термінових заявок та зміну статусів фінансових зборів. Усі тести підтвердили коректну роботу реалізованих функцій.

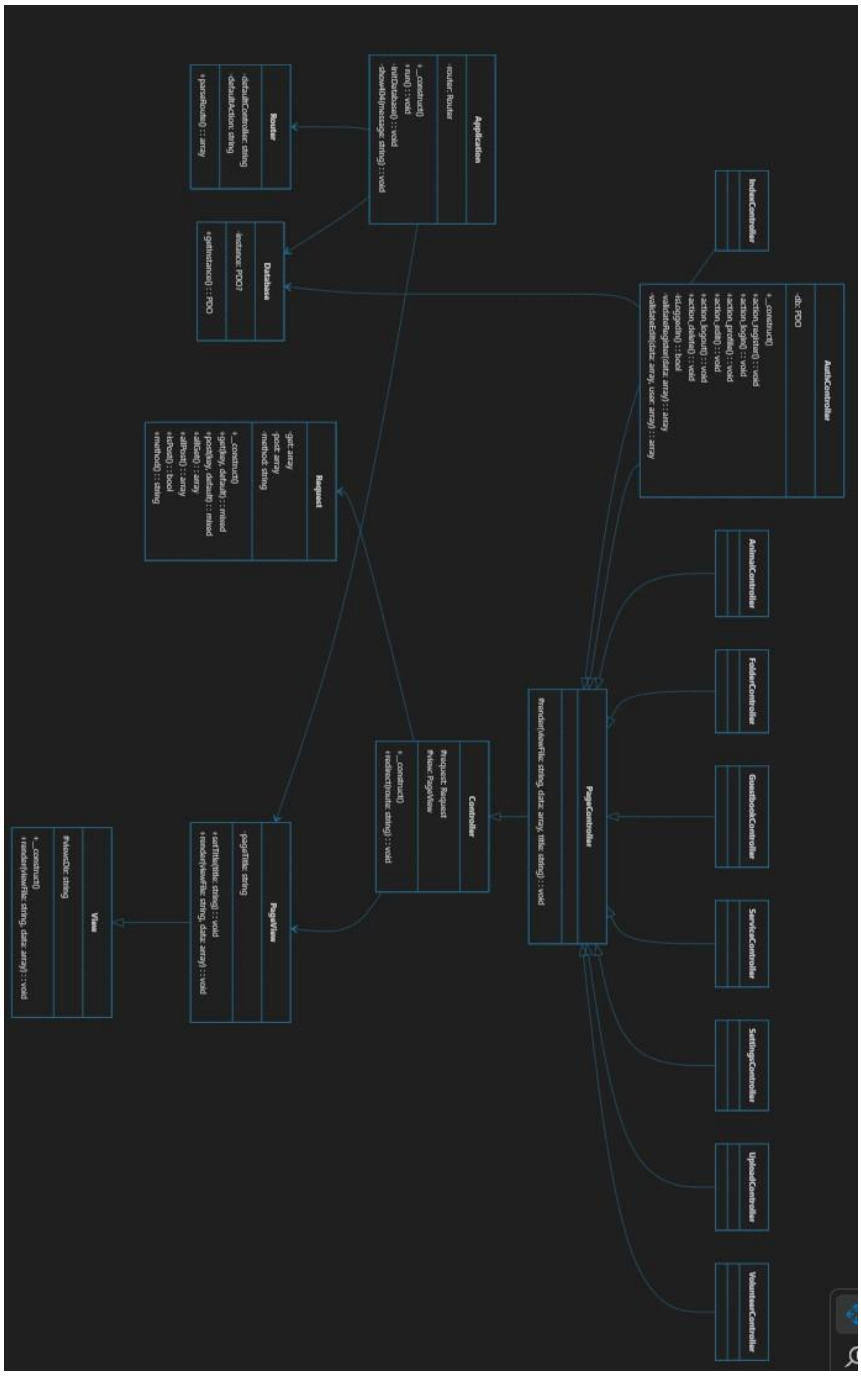
Таким чином, мету курсової роботи досягнуто — розроблено повнофункціональний вебзастосунок ветеринарної клініки, який не поступається функціоналом наявних аналогів за ключовими функціональними характеристиками та може бути використаний як основа для реального практичного застосування.

СПИСОК ЛІТЕРАТУРИ

1. PHP Tutorial. W3Schools. URL: https://www.w3schools.com/php/
2. PHP: Hypertext Preprocessor : Official Documentation. URL: https://www.php.net/docs.php (date of access: 15.09.2026).
3. PHP: Hypertext Preprocessor : Official Documentation. URL: https://www.php.net/docs.php (date of access: 15.09.2026).
4. SQLite Documentation : File-Based Relational Database Management System. URL: https://www.sqlite.org/docs.html (date of access: 12.09.2026).
5. W3C CSS Cascading Style Sheets Home Page / World Wide Web Consortium. URL: https://www.w3.org/Style/CSS/ (date of access: 18.09.2026).
6. PDO Drivers and Prepared Statements in PHP : Official Manual. URL: https://www.php.net/manual/en/pdo.prepared-statements.php (date of access: 10.09.2026).
7. MDN Web Docs : HTTP Authentication and Session Management. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication (date of access: 14.09.2026).
8. Visual Studio Code Documentation : PHP and Web Development Workflows. URL: https://code.visualstudio.com/docs/languages/php (date of access: 08.09.2026
9. Bootstrap Documentation : Design System and Responsive Layouts. URL: https://getbootstrap.com/docs/5.3/getting-started/introduction/ (date of access: 11.09.2026).
10. Bootstrap Documentation : Design System and Responsive Layouts. URL: https://getbootstrap.com/docs/5.3/getting-started/introduction/ (date of access: 11.09.2026).

ДОДАТКИ

Додаток А



Діаграма класів

Додаток Б

Лістинг методу SettingsController.php:

```
<?php
class SettingsController extends PageController
{
public function action_color(): void
{
$theme = $this->request->get('theme', 'light');
$fontSize = $this->request->get('font_size', 'normal');
$highContrast = $this->request->get('contrast', 'off');
$_SESSION['ui_theme'] = in_array($theme, ['light', 'dark', 'solarized'], true) ?
$theme : 'light';
$_SESSION['ui_font_size'] = in_array($fontSize, ['small', 'normal', 'large'],
true) ? $fontSize : 'normal';
$_SESSION['ui_contrast'] = ($highContrast === 'on');
$referer = $_SERVER['HTTP_REFERER'] ?? 'home/index';
$this->redirectUrl($referer);
}
public function action_greeting(): void
{
$name = trim($this->request->post('user_name', ''));
$gender = $this->request->post('user_gender', 'neutral');
if ($name !== '') {
setcookie('pref_user_name', $name, time() + (86400 * 30), '/');
setcookie('pref_user_gender', $gender, time() + (86400 * 30),
'/');
}
$this->redirect('home/index');
}
}
```

Лістинг методу UploadController.php:

```
<?php
class UploadController extends PageController
{
private string $uploadDir;
private array $allowedTypes = ['image/jpeg', 'image/png', 'image/gif',
'image/webp'];
private int $maxFileSize = 5242880; // 5MB
public function _construct()
{
parent::_construct();
$this->uploadDir = DATA_DIR . '/uploads/'; if (!is_dir($this->uploadDir)) {
mkdir($this->uploadDir, 0777, true);
}
}
public function action_index(): void
{
$this->handleUpload('general');
}
public function action_emergency(): void
{
$this->handleUpload('emergency');
}
public function action_home(): void
{
$this->handleUpload('home_visit');
}
private function handleUpload(string $category): void
```

```
{
$errors = [];
$successMessage = '';
if ($this->request->isPost() && isset($_FILES['photo'])) {
$file = $_FILES['photo'];
if ($file['error'] !== UPLOAD_ERR_OK) {
$errors[] = 'Помилка завантаження файлу.';
} elseif ($file['size'] > $this->maxFileSize) {
$errors[] = 'Файл занадто великий (макс. 5MB).';
} else {
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mimeType = $finfo->file($file['tmp_name']);
if (!in_array($mimeType, $this->allowedTypes, true)) {
$errors[] = 'Недозволений тип файлу. Дозволені: JPG, PNG,
GIF, WebP.';
} else {
$ext = pathinfo($file['name'], PATHINFO_EXTENSION);
$filename = $category . '_' . time() . '_' . bin2hex(random_bytes(4)) . '.' .
$ext;
$targetPath = $this->uploadDir . $filename;
if (move_uploaded_file($file['tmp_name'], $targetPath)) {
$successMessage = 'Фото успішно завантажено!';
} else {
$errors[] = 'Не вдалося зберегти файл.';
}
}
}
}
$this->render('upload/index', [ 'errors' => $errors, 'successMessage' =>
$successMessage, 'images' => $this->getImages(), 'category' => $category,
], 'Завантаження фото');
}
private function getImages(): array
{
if (!is_dir($this->uploadDir)) { return [];
}
$files = scandir($this->uploadDir);
$images = [];
foreach ($files as $file) {
if ($file !== '.' && $file !== '..' && is_file($this->uploadDir .
$file)) {
$images[] = $file;
}
}
return $images;
}
}
```

Лістинг модуль волонтерства та збору коштів VolunteerController.php:

```
<?php
class VolunteerController extends PageController
{
private PDO $db;
public function _construct()
{
parent::_construct();
$this->db = Database::getInstance();
}
public function action_index(): void
{
$volunteers = [];
```

```
$fundraisers = []; try {
$stmt = $this->db->query('SELECT * FROM volunteers WHERE status = "approved" ORDER
BY created_at DESC');
$volunteers = $stmt->fetchAll() ?: [];
} catch (PDOException $e) {
}
try {
$stmt = $this->db->prepare(
'SELECT * FROM fundraisers WHERE status = :status ORDER BY
created_at DESC'
);
$stmt->execute([':status' => 'active']);
$fundraisers = $stmt->fetchAll() ?: [];
} catch (PDOException $e) {
}
$this->render('volunteer/index', [ 'volunteers' => $volunteers, 'fundraisers' =>
$fundraisers,
], 'Волонтери');
}
public function action_signup(): void
{
$message = '';
$errors = [];
if ($this->request->isPost()) {
$data = [
'name' => trim($this->request->post('name', '')), 'email' => trim($this->request-
>post('email', '')), 'phone' => trim($this->request->post('phone', '')),
'position' => $this->request->post('position', ''),
'experience' => trim($this->request->post('experience', '')), 'availability' =>
trim($this->request->post('availability',
'')) ?: 'За узгодженням',
];
$errors = $this->validateSignup($data); if (empty($errors)) {
try {
$stmt = $this->db->prepare(
'INSERT INTO volunteers (name, email, phone, position, experience, availability,
status)
VALUES (:name, :email, :phone, :position,
:experience, :availability, :status)'
);
$stmt->execute([
':name' => $data['name'],
':email' => $data['email'],
':phone' => $data['phone'], ':position' => $data['position'], ':experience' =>
$data['experience'],
':availability' => $data['availability'], ':status' => 'pending',
]);
$message = 'Дякуємо за вашу заяву! Ми скоро з вами зв\'яжемося для
підтвердження.';
} catch (PDOException $e) {
$errors['general'] = 'Помилка при взятті: ' . $e-
>getMessage();
}
}
}
$this->render('volunteer/signup', [ 'message' => $message, 'errors' => $errors,
], 'Стати волонтером');
}
public function action_donate(): void
{
$id = (int)$this->request->get('id', 0);
```

```
$stmt = $this->db->prepare('SELECT * FROM fundraisers WHERE id = :id AND status =
:status');
$stmt->execute([':id' => $id, ':status' => 'active']);
$fundraiser = $stmt->fetch();
if (!$fundraiser) {
$this->redirect('volunteer/index'); return;
}
$errors = [];
if ($this->request->isPost()) {
$amount = trim((string)$this->request->post('amount', ''));
$normalizedAmount = str_replace(',', '.', $amount);
if ($amount === '' || !is_numeric($normalizedAmount) || (float)$normalizedAmount
<= 0) {
$errors['amount'] = 'Вкажіть суму допомоги більше 0 грн.';
} elseif ((float)$normalizedAmount > 1000000) {
$errors['amount'] = 'Сума допомоги не може перевищувати 1 000
000 грн.';
}
if (empty($errors)) { try {
$stmt = $this->db->prepare(
'UPDATE fundraisers
SET collected_amount = MIN(target_amount, collected_amount + :amount)
WHERE id = :id AND status = :status'
);
$stmt->execute([
':amount' => (float)$normalizedAmount, ':id' => $id,
':status' => 'active',
]);
$_SESSION['flash_success'] = 'Дякуємо! Вашу допомогу
додано до збору.';
$this->redirect('volunteer/index'); return;
} catch (PDOException $e) {
$errors['general'] = 'Не вдалося зберегти допомогу.
Спробуйте ще раз.';
}
}
}
$this->render('volunteer/donate', [ 'fundraiser' => $fundraiser, 'errors' =>
$errors,
], 'Допомогти тваринi');
}
private function validateSignup(array $data): array
{
$errors = [];
if ($data['name'] === '') {
$errors['name'] = "Ім'я є обов'язковим.";
}
if ($data['email'] === '') {
$errors['email'] = 'Email є обов\'язковим.';
} elseif (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
$errors['email'] = 'Email повинен бути коректним.';
}
if ($data['phone'] === '') {
$errors['phone'] = 'Телефон є обов\'язковим.';
} elseif (strlen($data['phone']) < 10) {
$errors['phone'] = 'Телефон мав мати мінімум 10 символів.';
}
```

```
if ($data['position'] === '') {
$errors['position'] = 'Виберіть посаду.';
}
if (!in_array($data['position'], ['опікун', 'організатор', 'соціальний'], true)) {
$errors['position'] = 'Невірна посада.';
}
return $errors;
}
}
```